Weekly Task:
Name : Narendra Kumar S
Date : 26-07-2025

1.Create a class BankAccount in Python with private attributes __accountno,__name, __balance.
Add
parameterized constructor
methods:
deposit(amount)
withdraw(amount)
set_accountno
get_accountno
set_name
get_name
get_balance()
set_balance()

Solution:

```
class BankAccount:
    def __init__(self, accountno, name, balance):
        self.__accountno = accountno
        self.__name = name
        self.__balance = balance


    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(f"Deposited ₹{amount}. New balance:
₹{self.__balance}")
        else:
            print("Invalid deposit amount.")

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            print(f"Withdrew ₹{amount}. Remaining balance:
₹{self.__balance}")
        else:
            print("Invalid or insufficient funds.")

    def set_accountno(self, accountno):
        self.__accountno = accountno
```

```
    def get_accountno(self):
        return self.__accountno

    def set_name(self, name):
        self.__name = name

    def get_name(self):
        return self.__name

    def set_balance(self, balance):
        if balance >= 0:
            self.__balance = balance
        else:
            print("Balance cannot be negative.")

    def get_balance(self):
        return self.__balance

acc = BankAccount(12345, "Narendra", 5000)
acc.deposit(2000)
acc.withdraw(1000)
print(acc.get_name())
print(acc.get_balance())
```

Output:

```
Deposited ₹2000. New balance: ₹7000
Withdrew ₹1000. Remaining balance: ₹6000
Narendra
6000
PS C:\Users\Narendrakumar\Downloads\website\website>
```

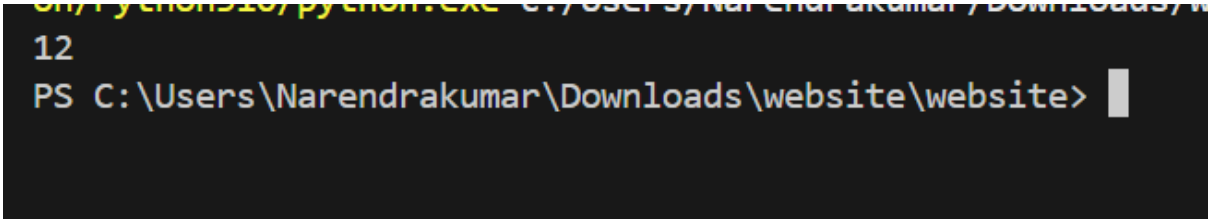
2.How will you define a static method in Python?Explore and give an example.

Solution:

A static method in Python is defined using the @staticmethod decorator. It does not receive the self or cls parameter and is used when a method doesn't depend on instance or class variables

```
class MathUtils:
    @staticmethod
    def add(a, b):
        return a + b
print(MathUtils.add(5, 7))
```

Output:



3.Give examples for dunder methods in Python other than __str__ and __init__ .

Solution:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

p1 = Point(1, 2)
p2 = Point(3, 4)
p3 = p1 + p2
print(p3.x, p3.y)
```

Output:

```
4 6
PS C:\Users\Narendrakumar\Downloads\website\website>
```

4. Explore some supervised and unsupervised models in ML.

Supervised Learning

1.Uses labeled data (input + output).
2.Goal: Predict output based on input.
3.Requires training with known results.
4.Example algorithms: Linear Regression, SVM.
5.Applications: Spam detection, price prediction.

Unsupervised Learning

1.Uses unlabeled data (no output).
2.Goal: Find hidden patterns or groups.
3.No predefined outcomes during training.
4.Example algorithms: K-Means, PCA.
5.Applications: Customer segmentation, anomaly detection.

5.Implement Stack with class in Python.

Solution:

```python
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)
        print(f"Pushed: {item}")

    def pop(self):
        if not self.is_empty():
            removed = self.items.pop()
            print(f"Popped: {removed}")
            return removed
        else:
            print("Stack is empty")
```

```
    def peek(self):
        if not self.is_empty():
            return self.items[-1]
        else:
            return None

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)

s = Stack()
s.push(10)
s.push(20)
print(s.peek())
s.pop()
print(s.size())
```

Output:

```
Pushed: 10
Pushed: 20
20
Popped: 20
1
PS C:\Users\Narendrakumar\Downloads\website\website>
```